

Simulating Your Embedded Project on Your Computer (Part 2)

by Nathan Jones



Mindstorms Engineering, LLC
Embedded Systems Design & Education

Contents

- [Introduction](#)
- [Virtual Hardware](#)
 - [Does the simulator support your microcontroller and/or the external components that you use in your system?](#)
 - [The "Main Major" Pattern](#)
 - [How easy is the simulator to learn and use?](#)
 - [How easy is it to incorporate building and running the simulation with your current build system?](#)
- [GUIs](#)
 - [Step 1: Assign the Layout](#)
 - [Step 2: Program the Event Handling](#)
 - [Step 3: Move Data Into/Out of the GUI](#)
 - [Communicating Between Different Executables](#)
- [A Near Miss: uC/Probe](#)
- [Summary](#)
- [Resources](#)

Part 1 of this series can be found [here](#).

Introduction

As [established in the last article](#), there are a tremendous number of compelling benefits to simulation. After ["isolating the madness"](#) (i.e. encapsulating any hardware-dependent code), the easiest means of simulating your embedded system is to simply replace all "outputs" with `printf` and all "inputs" with `getchar` (as part of a command-line interface of sorts). This is perfectly suitable but leaves a little to be desired from the simulator. In this article, I'll show you two additional means of simulation that, although potentially more difficult to set up, can either bring you much closer to a "real life" setup or greatly enhance the user experience when using the simulator. The first involves "virtual hardware": software that has been created to mimic real microcontrollers (and, sometimes, real circuits) as closely as possible. The second involves

creating a GUI, which looks a lot nicer than using the terminal and also gives us access to a few new input/output capabilities.

You can see working implementations of many of the techniques discussed in this and the previous article at [this Github repo](#).

Virtual Hardware

[Wokwi](#) is an example of what Jacob Beningo called, in a [recent article](#) on Design News, "virtual hardware": a piece of software that aims to actually mimic the code a microcontroller executes (and, for some, also any external electrical components connected to the microcontroller like LEDs, motors, LCD screens, etc). Examples of other virtual hardware simulators (besides Wokwi) are [TinkerCAD Circuits](#), [Renode](#), [Qemu](#), [Proteus](#), and [TINA](#). These tools are like a WYSIWYG document editor: if the simulator does something, you can reasonably expect your system to do that same thing in real life. When evaluating one of these tools, three questions become important:

1. Does the simulator support your microcontroller and/or the external components that you use in your system?
2. How easy is the simulator to learn and use?
3. How easy is it to incorporate building and running the simulation with your current build system?

Does the simulator support your microcontroller and/or the external components that you use in your system?

Let's address each separately. Regarding the microcontroller itself, it's not actually a deal-breaker if your microcontroller *isn't* supported (we *are* trying to set up our code so that it doesn't matter which processor it runs on, after all), but it's certainly nice if it does.

Regarding the specific external components that your system uses (e.g. LEDs, buttons, screens, sensors, motors, etc), if the simulator doesn't support them then you may be able to substitute them for something similar. For example, if the simulator doesn't have your specific I2C time of flight sensor, perhaps you can use a potentiometer or LDR instead; the substitute components would be sort of "mocking" the I2C sensor, since they would still provide the application code with an analog value to use in the simulation.

If the simulator doesn't support any external components (Renode and QEMU fall into this category), you might still be able to simulate inputs and outputs using `printf` and `scanf` (as discussed in the [last article](#)), provided the simulator at least lets you interact with your code through a virtual serial port.

The simulator holds little value if neither your microcontroller nor the external components you have in your system are supported, though. In that case, you might as well just compile your code to run on your computer and simulate inputs and outputs with `printf/scanf`.

The "Main Major" Pattern

Some simulators will also prescribe a development environment for the simulated microcontroller, such as requiring you to program in a simulated Arduino IDE. The problem with this is that the Arduino IDE already *has* a `main()` function (it's what calls `setup()` and `loop()`), meaning we need to find a way to hook our code into *that* main function and scrap the one we've written.

One solution to this is to use what John Taylor calls the "Main Major" pattern in his book ["Patterns in the Machine"](#): instead of having one `main()` function that calls a platform-dependent `initHardware()` function, we have multiple `main()` functions (one per platform) that each call the same function to run the actual application code.

For example, previously, our sample code looked like this:

```
*****
*      application.c      *
*****
void main(int argc, char ** argv)
{
    initHardware(argc, argv);
    while(1)
    {
        doStuff();
        doMoreStuff();
    }
}
*****
*      STM32.c            *
*****
void initHardware(int argc, char ** argv){...}
*****
*      x86.c              *
*****
void initHardware(int argc, char ** argv){...}
```

For the same code, the "Main Major" pattern would look like this:

```
*****
*      STM32.c            *
*****
void main(int argc, char ** argv)
```

```

{
    // Do STM32-specific HW initialization
    while(1) runTheApplication();
}

*****
*           x86.c           *
*****

void main(int argc, char ** argv)
{
    // Do x86-specific HW initialization
    while(1) runTheApplication();
}

*****
*   application.c   *
*****

void runTheApplication(void)
{
    doStuff();
    doMoreStuff();
}

```

Notice, now, that our application code is non-blocking (i.e. it's not inside it's own while(1) loop). This allows us to easily hook our application code in to whatever other main() function may already exist. If we were using a simulator that required us to program from inside the Arduino IDE, our code would just look like this:

```

void setup(void)
{
    // Do Arduino-specific HW initialization
}

void loop(void)
{
    runTheApplication();
}

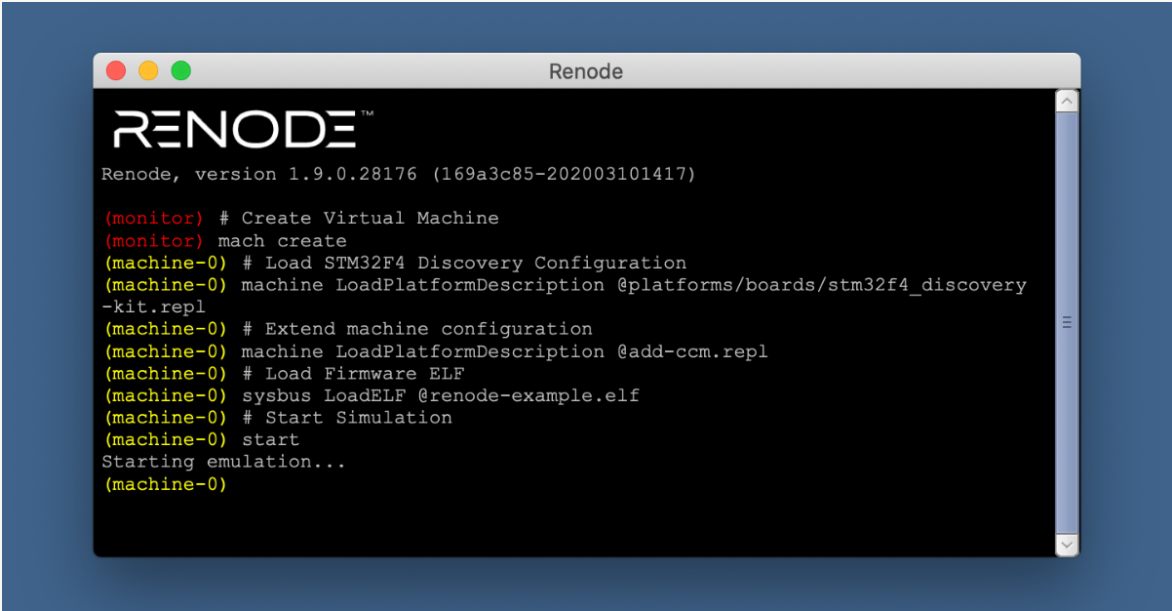
```

How easy is the simulator to learn and use?

This is important because, as I've mentioned, setting up and running your simulation should take only a fraction of the time it will take to develop your embedded project (otherwise we stand to lose any gains from using a simulation in the first place!). Although I don't have much experience with all of the simulators listed above, it would seem to me that the two simulators targeted towards the hobbyist/educational demographic (Wokwi and TinkerCAD) would be the easiest to use.

How easy is it to incorporate building and running the simulation with your current build system?

In other words, how difficult is it to run your code in your simulator after you've made any changes? To me, it would be ideal if I only needed to press a single button or run a single command from my terminal in order to rebuild my code for the simulator. Tools that run on the command line or that have command-line interfaces would have an edge here; of the simulators I listed above, that would be Renode and QEMU. Graphical simulators such as Wokwi or TinkerCAD, for all their ease of use, are at a disadvantage here. Updating your project in Wokwi or TinkerCAD requires uploading new source files, which isn't hard, but you'd have to remember to propagate any changes in the simulator *back* to your original source files and I don't even want to think about how you might manage your project if it relied on a lot of external libraries that were difficult to pull into the simulator. You run the real risk of having "the same information in two places, guaranteeing that one of them is always wrong", as one of my old coworkers liked to say.



```
Renode
RENODE™
Renode, version 1.9.0.28176 (169a3c85-202003101417)

(monitor) # Create Virtual Machine
(monitor) mach create
(machine-0) # Load STM32F4 Discovery Configuration
(machine-0) machine LoadPlatformDescription @platforms/boards/stm32f4_discovery-kit.repl
(machine-0) # Extend machine configuration
(machine-0) machine LoadPlatformDescription @add-ccm.repl
(machine-0) # Load Firmware ELF
(machine-0) sysbus LoadELF @renode-example.elf
(machine-0) # Start Simulation
(machine-0) start
Starting emulation...
(machine-0)
```

Wokwi *does* have a command-line interface (sort of), called "[Wokwi-cli](#)". This tool *will* run a Wokwi simulation but your ability to interact with it while it's running is limited to the serial interface or to using a predefined "[automation scenario](#)" (currently in alpha).

It seems to me that **Wokwi** and **Renode** hold the most promise (unless neither of these simulators supports your microcontroller but a different one does) and I think I'll be trying to learn how to use them both better in the near future.

GUIs

Let's revisit the idea of simulating our embedded project with printf/getchar. A natural progression to this kind of simulation is to create a GUI. Instead of entering values at the command line you could adjust knobs, buttons, sliders, and dials; instead of reading values

using printf you could read display panels, gauges, graphs, and indicator lights. Having such a thing would feel like having a dashboard or cockpit for your embedded system; you could simulate anything and have an unprecedented level of control over your embedded system!

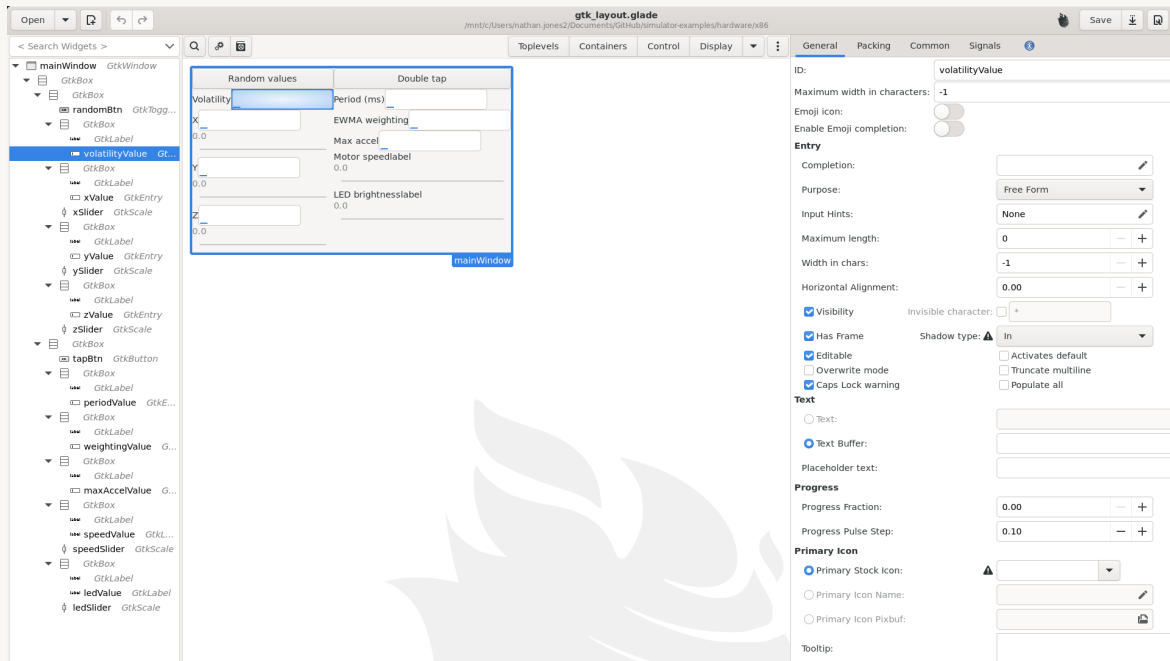


Unfortunately, most GUI frameworks strive to be powerful at the expense of simplicity, it seems, and the complexity of this type of simulation can be significantly higher than compared to one that simply uses printf/getchar. Or maybe its more accurate to say that GUI frameworks are written so that developers can make large, complex desktop applications with all the bells and whistles whereas what we want is just something that lets us read from and write to some variables, so the complexity ends up getting in the way. Either way, plan on creating a GUI to take more time than you think it will, at least until you become very comfortable with a specific GUI framework. The following GUIs may be easier to use than others, for reasons I'll discuss below.

GUI	Language	Layout manager	Relative placement	Mode
Qt	C++ / Python	QtCreator	Yes	Retained
GTK / gtkmm	C (GTK) C++ (gtkmm)	Glade	Yes	Retained
FlatUI	C++	None	Yes	Immediate
raygui	C	rGuiLayout	No	Immediate
PySimpleGUI	Python	None	Yes	Immediate

Step 1: Assign the Layout

Using any GUI, your first task will be to assign the layout of each and every widget (i.e. every text box, slider, button, etc). Some GUIs will require the programmer to specify the exact size and location of each widget when doing so, which can be rather tedious and may require lots of trial and error to make sure things line up correctly. To mitigate this, some GUIs have layout managers that let a programmer drag and drop widgets to create their desired layout and then generate some kind of design file based on that layout; the GUI application is then programmed to consume this file and automatically build the desired layout. [Glade](#) for GTK/gtkmm, [QtCreator](#) for Qt, and [rGuiLayout](#) for raygui, are examples of layout managers.



A screenshot showing the Glade Layout Manager for GTK. Widgets can be added to, moved around, and deleted from the main window in the middle of the Glade tool and a file can then be generated to easily create the desired layout in GTK.

Other GUIs eschew the exact positioning of each widget (possibly allowing it as an optional argument) in favor of relative placement: if a vertical layout has been assigned and the code creates widgets A, B, and C (in that order), then the GUI draws widget A above widget B, which is drawn above widget C. All of the GUIs listed above except for raygui (i.e. Qt, GTK/gtkmm, FlatUI, and PySimpleGUI), allow for the relative placement of widgets. Here's an example from PySimpleGUI that creates a window with two labels, a text entry, and two buttons.

```
# PySimpleGUI code sample
layout = [ [sg.Text('Some text on Row 1')],
            [sg.Text('Enter something on Row 2'), sg.InputText()],
            [sg.Button('Ok'), sg.Button('Cancel')] ]
```

When [run](#), this code produces the window below.



Step 2: Program the Event Handling

Next, you'll need to describe how the GUI should respond to events (and which events it should respond to). To some extent, every GUI will automatically respond to user events: text entries will show characters that are typed, sliders will let themselves be dragged to one side or the other, buttons will slightly change their shading when clicked, etc. What happens once those elements have been changed, though, must be specified by the programmer. Here, a distinction must be drawn between "retained mode" and "immediate mode" GUIs. For retained mode GUIs, the programmer creates a layout once and then specifies how events should be handled by connecting each event (e.g. button clicked, button released, text entry in focus, text entry changed, text entry finished, etc) to a specific function that gets called when that event occurs. In the GTK code below (taken from [here](#)), a "clicked" event on the quitButton widget will result in the function quitButtonClicked being called.

```
// GTK code sample
static void quitButtonClicked(GtkWidget *widget, gpointer data);
int main(int argc, char **argv)
{
    // Make the window
    GtkWidget *window = gtk_window_new(GTK_WINDOW_TOPLEVEL);
    // Make the button
    GtkWidget *quitButton = gtk_button_new_with_label("QUIT");
    // Add the button to the window
    gtk_container_add(GTK_CONTAINER(window), quitButton);
    // Register the button callback
    g_signal_connect(quitButton, "clicked", G_CALLBACK(quitButtonClicked), NULL);
}
```

This is, I think, the more common type of GUI. In an **immediate mode GUI**, on the other hand, event-handling happens in conjunction with the widget itself being created: the same function that creates a button will return a Boolean value indicating whether the button is currently being pressed (or an enumeration for which click event occurred on the button, like in the FlatUI code sample below).

```
// FlatUI code sample
gui::StartGroup(gui::kLayoutVerticalLeft, 5);
    gui::Label("value: " + x, 40);
```



```
if (gui::TextButton("increase", 40) == kEventWentUp) x++;  
gui::EndGroup();
```

These functions are called in the main loop of an immediate mode GUI which runs as fast as the frame rate (or faster), meaning that events get handled (and the GUI updated) tens or hundreds of times a second. The result is, to me, *orders of magnitude* simpler and easier to understand than using a sea of callbacks in a retained mode GUI. FlatUI, raygui, and PySimpleGUI are examples of immediate mode GUIs. (FWIW, it also seems possible to program a retained mode GUI *as if* it were an immediate mode GUI by eschewing any callbacks and simply checking whether any pertinent events have occurred using one of the three options below.)

What do you do if your GUI needs to complete some piece of work that isn't directly triggered by a user interaction? (A good example of this might be our application code! If the GUI has a main loop that we can't directly edit then we'll likely need to use the [Main Major](#) pattern, effectively making our application code a separate "task" within the GUI.) With a retained mode GUI, you have three options:

- Put the work to be done inside a timer callback that runs at a specific rate
- Put the work inside a thread that gets spawned when the GUI is created
- Put the work inside the GUI's idle loop

For example, in [this GUI](#) I needed to respond to messages that showed up on a serial port. I choose the simplest option of creating a [timer callback](#) that ran every 10 ms and which [read from the serial port](#) if there were any characters on it.

If the work to be done by a retained mode GUI gets triggered by another piece of software that you control, then you can also create a queue and an associated callback in the GUI to process messages/events posted to the queue.

On the other hand, when an immediate mode GUI needs to do something that isn't directly triggered by a user action, you simply add that code before or after the GUI code. The GUI code is non-blocking and this allows the main loop to perform the additional work at the same time as managing the GUI. Instead of needing to shoe-horn our application code into a timer callback or idle loop, you can simply have the [main loop execute it right before](#) or after it handles the GUI. Yet another reason why I think immediate mode GUIs are so much simpler to program with than retained mode GUIs!

Step 3: Getting Data Into/Out of the GUI

Since we're using the GUI to simulate the hardware parts of our project, we'll also need to figure out how to get data into and out of our GUI (simulating the reading from and writing to hardware peripherals). Here, we need to make a distinction between GUIs that are running in the same executable as our application code versus GUIs that aren't. As long as our GUI is running in the same executable as our application code, we have the typical tools available to us for sending data between threads/modules: global variables, mailboxes/queues, etc.

The simplest solution is to use an immediate mode GUI and to make the elements of that GUI that the application code needs to read from/write to global (or, rather, "file local"). If we do that, then the hardware-dependent functions that the application calls can simply [modify those local variables](#); an immediate mode GUI will natively take those new values and update itself every time its main loop runs.

```
*****
*   raygui_advanced.c   *
*****
float xValue = 0.0f;
float yValue = 0.0f;
float zValue = 0.0f;
float motorSpeedSliderValue = 0.0f;

...
int main()
{
    ...
    while (!WindowShouldClose())
    {
        ...
        GuiSlider((Rectangle){ 24, 128, 120, 16 }, NULL, NULL, &xValue, -10, 10);
        GuiSlider((Rectangle){ 24, 176, 120, 16 }, NULL, NULL, &yValue, -10, 10);
        GuiSlider((Rectangle){ 24, 224, 120, 16 }, NULL, NULL, &zValue, -10, 10);
        ...
        GuiSliderBar((Rectangle){ 184, 168, 120, 16 }, NULL, NULL,
&motorSpeedSliderValue, -1, 1);
        ...
    }
    ...
}
void readAccel_gs(double* x, double* y, double* z)
{
    ...
    *x = (double)xValue;
    *y = (double)yValue;
    *z = (double)zValue;
}
void setMotorSpeed(double speed)
{
    motorSpeedSliderValue = (float)speed;
}
```

If you insist on using a queue, though, or if you're using a retained mode GUI, things get a little more complex. The GUI won't automatically update itself, so it has to be programmed to check for new data, which amounts to additional work that the GUI needs to do. This "checking for new data" will need to be put into either a timer callback, a worker thread, or the GUI's idle loop, as

mentioned above (unless we can make a custom event for the GUI to respond to). The interaction between the GUI and the application code also gets a bit complex in this case: the application code can't send or receive data *immediately* from the simulated hardware, it can only, effectively, make *requests* for new data to be sent or received and then wait for the "hardware" (i.e. the GUI) to process that request. An example of this is shown below, which was taken from [this project](#).

```
void* readThread(void* data)
{
    while(1)
    {
        // Get values from the GUI
        switch(argv[0][0])
        {
            case 'a':
                curr_x = atof(argv[1]);
                curr_y = atof(argv[2]);
                curr_z = atof(argv[3]);
                new_vals = true;
                break;
        }
    }
}

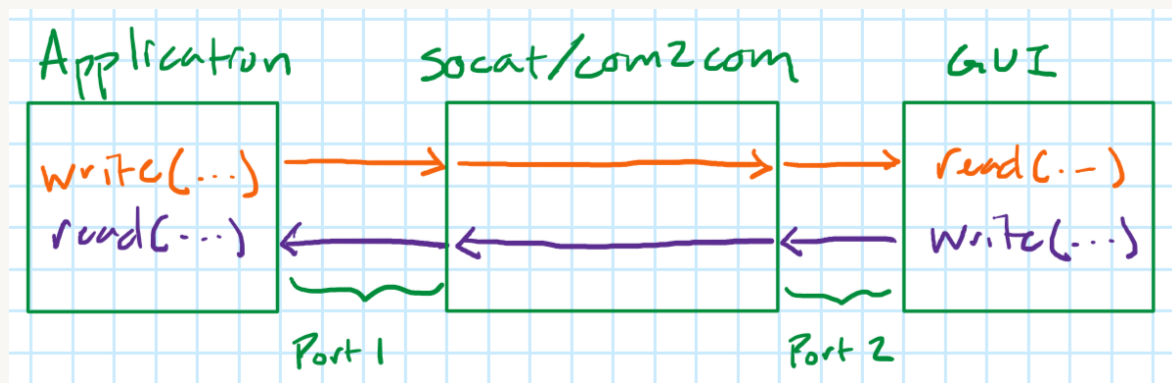
void readAccel_gs(double* x, double* y, double* z)
{
    // Send out "r" command to request updated accel values
    write(serial_port, "r\n", 2);
    while(!new_vals);
    *x = curr_x;
    *y = curr_y;
    *z = curr_z;
    new_vals = false;
}
```

The application code *can't* get new acceleration data synchronously. Instead it makes a request for new data (using a virtual serial port; see below) and waits for a flag (`new_vals`) to be set. Once the data arrives from the GUI, the three local variables are updated and the function that was called by the application code is allowed to return.

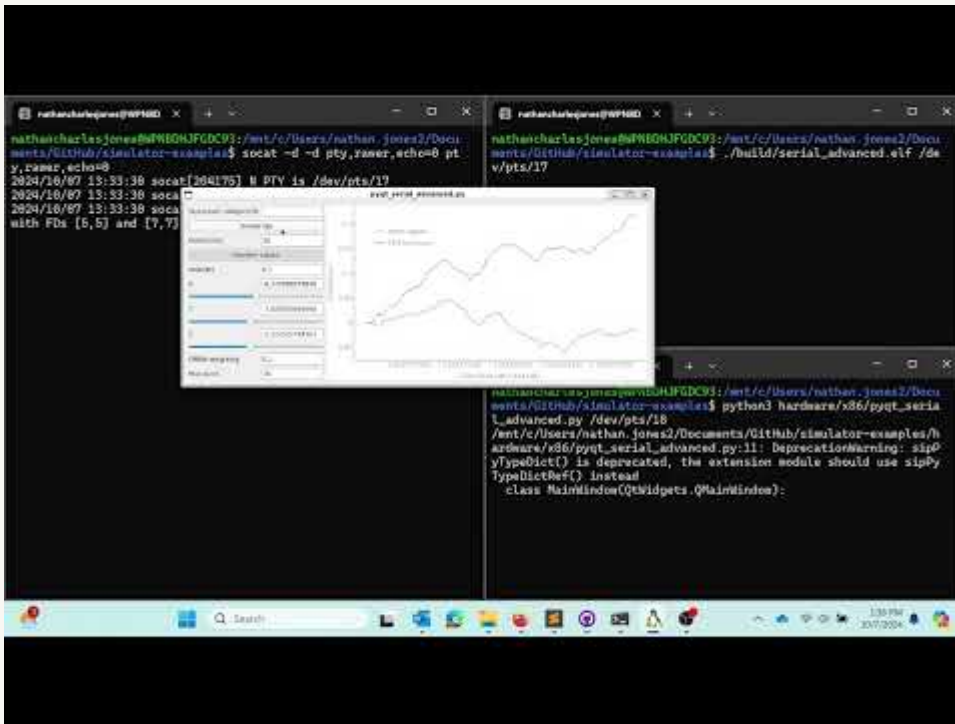
Communicating Between Different Executables

Believe it or not, its entirely possible to move data into or out of our GUI even if it's not running in the same executable as our application code! This is particularly helpful if you want to write your GUI in a completely different programming language (e.g. Python) than your application code is written in.

The problem, in essence, is that you would want to have two separate programs running simultaneously on your computer: one that is your application and the other that is your GUI. They need to talk to each other, but they can't do so simply by means of a function call (like they could if they were in the same application). The trick, then, is to create a **virtual serial port** and then have your two programs read from and write to that serial port (here I'm using a "plain" serial port, but you could also use something like a TCP socket). We can use [socat](#) on Linux (run `socat -d -d pty,rawer,echo=0 pty,rawer,echo=0` from a shell) or [com2com](#) on Windows to accomplish this. Either tool will actually create two serial ports, one for each side of the communication channel. (Neither OS will let two programs control the same serial port so, instead, socat and com2com each create two ports and, internally, route the traffic from one to the other.)



In the hardware-dependent code inside your embedded project, you'll essentially just replace `printf/scanf` with `write/read`. In your GUI framework of choice, you'll only need to add code that reads from a serial port at some regular interval (or whenever a byte is received, if your GUI has support for that). In my [example Git repo](#), I wrote a GUI in PyQt to mimic an accelerometer, motor, and LED. The application requests new acceleration values by sending the message "r\n" across the virtual serial port to the GUI and the GUI (when it eventually processes any received messages) then responds with "a \n", as in the sample code above. Elsewhere, the application is able to set a new motor speed by sending the message "m \n". In the PyQt GUI, I check the serial port every 10 ms and [process any messages on the serial port](#).



<https://www.youtube.com/watch?v=c46W2OUhtFo>

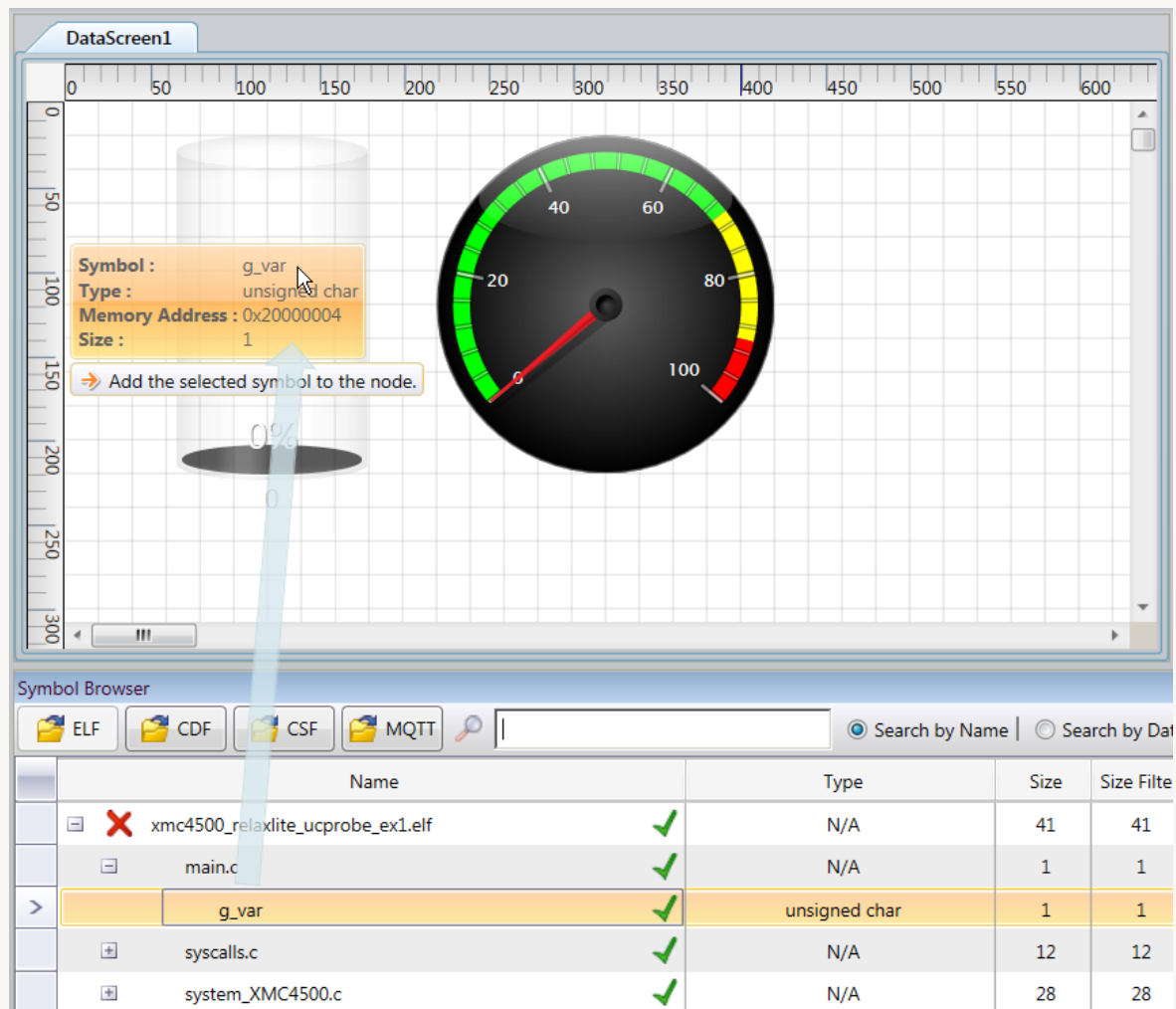
A Near Miss: uC/Probe



<https://www.youtube.com/watch?v=BFR5salinDY>

A GUI looks cool but feels like overkill when all we really want to do is read/write data from our system. It would be nice if there was a GUI framework that could parse our program and let us assign widgets to directly read or control our program's data: toggle buttons get connected to a Boolean value so that they change from true to false when the button is pressed, sliders get connected to a float value, etc. In fact a program like this did (does?) exist: uC/Probe! It was originally developed to simply observe embedded systems and the easiest way to set it up

required the use of a debug adapter like a J-Link, but it *also* included the ability to send data over a serial connection (instead of a J-Link) and I don't see why that wouldn't work with virtual serial links like we discussed in the last section.



Alas, I'm unsure of the state of this project after Micrium was bought by SiLabs, though it *may* still be available for in Simplicity Studio (for specifically Silicon Labs MCUs). If you have an information about the availability of this awesome piece of software, post it in the comments!

[FreeMASTER](#) (for NXP/Freescale MCUs) and [STM32Monitor](#) (for STM32 MCUs) are similar programs with seemingly the same capability of communicating to a target MCU over serial. Whether or not these might also work would seem, to me, to depend on whether or not those programs expect or require the processor they're communicating with to be an NXP/Freescale or STM32 processor.

Summary

Simulating your embedded project isn't just a neat thing to do, it's a "superpower" (like Uri Shaked said), allowing you to:

- develop and test code anywhere,
- separate software bugs from hardware bugs,
- easily adjust to changing project requirements,

- use desktop-only applications like Valgrind or reverse debugging to aid in development, and
- observe and control your system when it eventually does run on real hardware (possibly even remotely!).

The easiest way to simulate your embedded system is with `printf/getchar` (or `fprintf/fgetc`). It's not super flashy, but it gets the job done.

Using virtual hardware gets you closest to the "real deal", but you'll need to consider whether it supports your MCU and hardware peripherals, easily integrates into your build system, and is easy to learn, to decide if it's worth the effort.

Creating a GUI makes for a cleaner user experience than using `printf/getchar`, but there are plenty of hurdles to overcome and you should expect it to take longer than you think it should at first (at least, until you get very comfortable programming with a specific GUI framework). I think immediate mode GUIs like FlatUI, raygui, and PySimpleGUI are simpler and easier to work with than retained mode GUIs like Qt or GTK/gtkmm.

I hope this two-part series has helped you understand how to simulate your next embedded project. Throw any questions my way in the comments. And happy hacking!

Resources

- Renode
 - <https://interrupt.memfault.com/blog/intro-to-renod...>
 - <https://medium.com/@pc0is0me/getting-started-with-...>
- GTK/gtkmm
 - <http://www.peteronion.org.uk/GtkExamples/GladeTuto...>
 - <https://gnome.pages.gitlab.gnome.org/gtkmm-documen...>
- Software serial
 - <https://github.com/gbmhunter/CppLinuxSerial>
 - <https://www.baeldung.com/linux/make-virtual-serial...>
 - <https://copyconstruct.medium.com/socat-29453e9fc8a...>
 - <https://com0com.sourceforge.net/>